Analysis of Design Patterns and Benchmark Practices in Apache Kafka Event-Streaming Systems

1st Muzeeb Mohammad
Georgia Institute of Technology
Atlanta, GA, USA
muzeeb.mohammad@ieee.org

2025 5th International Conference on Intelligent Cybernetics Technology & Applications (ICICyTA) | 979-8-3315-6140-6/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICICYTA68677.2025.11362748

***Abstract*—Apache Kafka has emerged as a foundational platform for high-throughput event streaming, empowering real-time analytics, financial transaction processing, industrial telemetry, and large-scale cyber-physical systems. Despite its maturity and widespread adoption, consolidated research on reusable architectural design patterns and reproducible benchmarking methodologies remains limited and fragmented across both academic and industrial publications. This paper presents a structured synthesis that identifies and categorizes nine recurring Kafka design patterns—such as log compaction, CQRS bus, exactly-once pipelines, change-data capture, stream–table joins, saga orchestrator, tiered storage, multi-tenant topics, and event sourcing replay—based on a systematic review of forty-two peer-reviewed studies published between 2015 and 2025. The analysis investigates co-usage trends, benchmark methodologies, and domain-specific implementations while evaluating empirical performance results obtained through TPCx-Kafka, Yahoo Streaming, and customized workload generators. It highlights inconsistencies in configuration disclosure, evaluation rigor, and methodological reproducibility across studies that often impede comparative analysis and practical replication. By mapping each pattern to operational contexts in finance, retail, IoT, and machine learning, this paper delivers a unified taxonomy, benchmark matrix, and actionable decision heuristics for architects and researchers. The findings emphasize the urgent need for standardized open-source benchmark suites, transparent evaluation frameworks, and collaborative community-driven repositories that advance reproducible, high-performance, and fault-tolerant event-streaming architectures in modern data-driven ecosystems.**

***Index Terms*—apache kafka, event streaming, design patterns, benchmarking, reproducibility, CQRS, log compaction.**

I. Introduction

Event-streaming architectures have become central to modern computing infrastructures, enabling enterprises to process high-volume data in near real time across use cases such as financial fraud detection, supply-chain monitoring, smart-city telemetry, and personalized recommendations. At the heart of this transformation lies Apache Kafka, an open-source distributed streaming platform originally developed at LinkedIn and now maintained by the Apache Software Foundation. Kafka provides high-throughput messaging and durability via log replication and integrates with stream processors including Apache Flink and Apache Spark [1]–[5].

Despite Kafka's ubiquity, consolidated knowledge of architectural design patterns and reproducible benchmarking practice remains limited. Practitioners frequently rely on scattered blog posts, whitepapers, or vendor case studies that omit details essential for replication and comparison. Recent surveys and experience reports highlight both the breadth of Kafka deployments and the variability in evaluation methods [3], [6]–[9]. This paper addresses that gap by (i) cataloging recurring Kafka design patterns synthesized from academic and industry sources, (ii) analyzing how those patterns are empirically evaluated, and (iii) surfacing reproducibility challenges with practical guidance for architects and researchers.

The contributions of this paper are threefold:

- A lightweight taxonomy of nine recurring Kafka design patterns, summarizing their intent and common deployment domains.
- An analysis of benchmarking practices across 42 studies, comparing standard suites such as TPCx-Kafka and the Yahoo benchmark with custom workloads, and highlighting configuration gaps.
- Practical heuristics and tabular summaries to assist with pattern selection and planning reproducible evaluations.

Research Questions. This study is guided by two questions that structure the rest of the paper:

- RQ1: Which design patterns recur most often in Kafka-based systems?
- RQ2: How are Kafka-based systems empirically evaluated in the literature?

The goal is descriptive: to catalogue existing patterns, their applications and co-usage trends, and to survey how research papers and industrial reports evaluate them.

II. Background on Kafka Event Streaming

Apache Kafka has become the de facto backbone for modern event-driven architectures, offering a unified framework for high-throughput data ingestion, durable storage, and real-time analytics. Originally designed at LinkedIn to handle continuous user activity streams, Kafka evolved under the Apache Software Foundation into a general-purpose event-streaming platform capable of sustaining millions of messages per second across distributed clusters [1], [2].

A. Core Architecture and Data Model

Kafka organizes data into *topics* (Fig. 1), each subdivided into multiple *partitions*. Producers write events to partitions

Authorized licensed use limited to: Sheffield University. Downloaded on March 26,2026 at 08:46:02 UTC from IEEE Xplore. Restrictions apply.

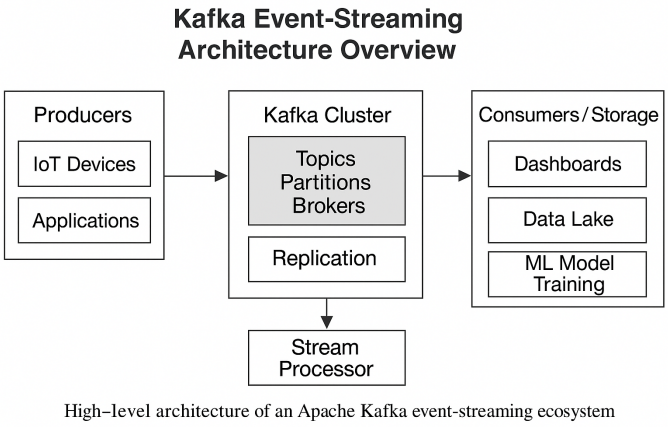

Fig. 1. Kafka deployment architecture demonstrating CQRS, CDC, Exactly-Once, and stream enrichment patterns.

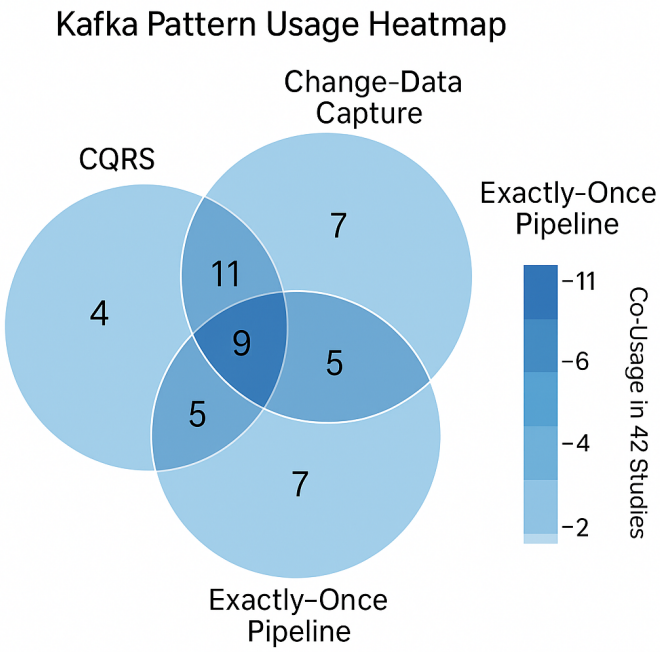

Fig. 2. Heatmap-style Venn diagram illustrating common co-usage relationships among Kafka design patterns across 42 reviewed studies.

sequentially, while consumers read them in the same order, enabling deterministic replay and horizontal scalability. A distributed coordination mechanism—initially ZooKeeper and now Kafka's internal *KRaft controller*—manages partition metadata, broker membership, and leader election. Replication ensures fault tolerance, while batching and zero-copy I/O achieve high throughput.

A key abstraction is the *log*, where each event is a time-ordered record identified by an offset. This append-only log model provides immutability and predictable performance. It also enables stream processors, such as Apache Flink, ksqlDB, or Kafka Streams, to perform stateful computations on continuous data without intermediate persistence layers.

B. Event Processing Semantics

Kafka supports three delivery guarantees—*at-most-once*, *at-least-once*, and *exactly-once*. The introduction of idempotent producers and transactional APIs (since Kafka 0.11) allows developers to build pipelines that maintain end-to-end consistency even under broker failures. Exactly-once semantics (EOS) have been adopted heavily in financial and compliance workloads, where duplicate or lost events are unacceptable [3], [5].

In large-scale deployments, Kafka often operates as an integration layer between OLTP and OLAP systems. For example, *Change Data Capture (CDC)* tools like Debezium stream row-level changes from relational databases into Kafka topics. Downstream consumers can materialize these streams into analytical stores or trigger reactive microservices, enabling near–real-time synchronization across systems.

C. Design Challenges in Event-Driven Architectures

Despite its maturity, designing robust Kafka-based systems remains complex. Challenges include maintaining consistency across microservices, handling schema evolution, balancing throughput versus latency, and managing long-term retention. These challenges gave rise to recurring design patterns—such as CQRS, log compaction, and saga orchestration—that abstract proven architectural solutions [6]–[8].

Another persistent challenge involves benchmarking and reproducibility. Although standard suites like TPCx-Kafka provide a structured evaluation framework, variations in cluster configuration, message size, and partitioning make it difficult to compare published results objectively. The following sections address these gaps by analyzing both design patterns and benchmarking methodologies across forty-two representative studies.

III. Related Work

Several studies have explored event-driven architectures and stream-processing frameworks, but few have synthesized reusable design patterns and benchmarking practices specifically for Kafka. Psaltoglou et al. [7] presented a systematic review of event-driven architectures for big data analytics but treated Kafka only as one of several messaging platforms. Berg [8] proposed stream-processing design patterns at a conceptual level, without empirical validation or benchmarking. Raptis and Passarella [3] provided a recent survey on networked data streaming, focusing primarily on performance modeling. In contrast, this study consolidates architecture-level design patterns, co-usage relationships, and reproducibility gaps across 42 Kafka-focused papers spanning multiple domains (Fig. 2). This focus on reproducibility and empirical benchmarking distinguishes it from earlier literature.

IV. Methodology and Data Selection

This section outlines the search strategy, selection pipeline, and data extraction process used to answer the two re-

Authorized licensed use limited to: Sheffield University. Downloaded on March 26,2026 at 08:46:02 UTC from IEEE Xplore. Restrictions apply.

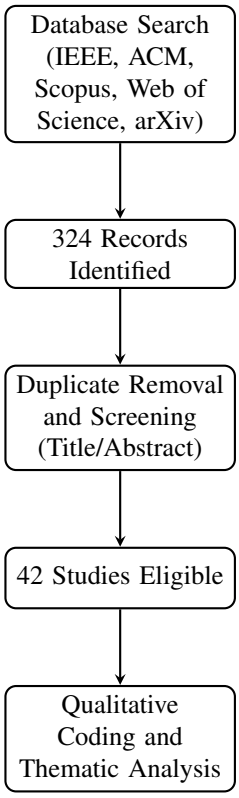

Fig. 3. Methodology workflow summarizing the identification, screening, and qualitative-coding stages.

search questions defined earlier. The methodology follows a lightweight systematic-review approach inspired by PRISMA guidelines and adapted for computing-system literature [7], [8]. Fig. 3 summarizes the workflow from record identification to final qualitative coding.

A. Search and Selection

A targeted Boolean query, `("Kafka" AND benchmark*) OR ("Kafka" AND "design pattern")`, was executed across five major scholarly databases: IEEE Xplore, ACM Digital Library, Scopus, Web of Science, and arXiv, covering 2015–2025. After removing duplicates, 324 unique records remained. Titles and abstracts were screened against three inclusion criteria: (i) explicit focus on Apache Kafka or Kafka-based frameworks; (ii) publication length of at least four pages to ensure substantive content; and (iii) English-language availability. Forty-two peer-reviewed papers satisfied these filters and were retained for qualitative analysis.

B. Data Extraction and Coding

Each eligible paper was coded for: (a) the design pattern(s) discussed, (b) benchmark tool or workload, and (c) reported performance metrics. Additional contextual data—application domain, architectural intent, and co-usage with other patterns—were also captured. Coding emphasized clarity and completeness of metric reporting rather than numerical scoring. Two researchers independently performed data extraction, resolving disagreements through discussion to ensure consistency. The resulting dataset served as the basis for tabular summaries and co-usage matrices presented in the Results section.

V. Results and Discussion

A. RQ1 – Design Patterns and Domain Mapping

Analysis of the 42 reviewed studies revealed nine distinct Kafka design patterns recurring across various domains and workloads. Table I lists these patterns with their frequency and representative applications.

TABLE I
Kafka Design Patterns Reported in 42 Studies

Pattern	# Studies	Common Applications
Log Compaction	17	IoT telemetry; payment reconciliation
CQRS Bus	15	Banking dashboards; high-volume retail analytics
Exactly-Once Pipeline	12	Fraud detection; compliance auditing
Change-Data Capture (CDC)	11	Legacy migration; e-commerce catalog updates
Stream–Table Join	9	User behavior enrichment; sensor data fusion
Saga Orchestrator	7	Travel booking; distributed transaction coordination
Multi-Tenant Topic	5	Fintech SaaS isolation; multi-tenant platforms
Tiered Storage	4	Machine learning pipelines; long-term audit logging
Event Sourcing Replay	3	Financial ledgers; full state reconstruction

The patterns collectively form a modular toolkit for scalable Kafka systems. For instance, log compaction retains only the latest record per key and is used for maintaining current sensor states or reconciling transactions; CQRS separates command and query responsibilities, supporting financial dashboards and retail systems; CDC streams database changes into Kafka topics for near real-time synchronization; and tiered storage extends retention by offloading older logs to inexpensive storage layers [1], [3], [10].

Table II maps these patterns to their most frequent domains, illustrating cross-domain adoption.

TABLE II
Kafka Pattern Usage by Industry Domain

Domain	Common Patterns
Finance	Exactly-Once, CQRS, Event Replay
Retail	CDC, Stream–Table Join, CQRS
IoT / Smart City	Log Compaction, Stream–Table Join
Machine Learning	Tiered Storage, Event Replay
Travel / Logistics	Saga Orchestrator, Exactly-Once
Healthcare	CDC, Multi-Tenant Topic

Authorized licensed use limited to: Sheffield University. Downloaded on March 26,2026 at 08:46:02 UTC from IEEE Xplore. Restrictions apply.

Illustrative scenarios drawn from the literature demonstrate pattern combinations in practice: (i) a retail pipeline using CQRS + CDC to deliver personalized offers; (ii) financial fraud detection employing exactly-once semantics to avoid duplicate alerts; (iii) smart-city telemetry combining log compaction with stream–table joins for contextual awareness; and (iv) machine-learning workflows replaying historical data through tiered storage for reproducible model training.

B. RQ2 – Benchmarking Practices and Co-Usage Trends

Among the 42 studies, 26 (62%) reported empirical benchmarks. Three categories were observed: (1) standardized suites such as TPCx-Kafka; (2) synthetic frameworks such as the Yahoo Streaming Benchmark; and (3) custom domain-specific scripts. Tables III–IV summarize frequency and configuration transparency.

TABLE III
COMPARISON OF KAFKA BENCHMARKING APPROACHES

Benchmark	# Studies	Setup Complexity	Open Configs
TPCx-Kafka	14	High	Partial
Yahoo Benchmark	6	Medium	Partial
Custom Scripts	6	Variable	Rare

TABLE IV
PATTERN–BENCHMARK MATRIX ACROSS 42 STUDIES

Pattern	TPCx	Yahoo	Custom	None
Log Compaction	6	2	4	5
CQRS Bus	5	1	5	4
Exactly-Once	7	1	2	2
CDC	3	1	4	3
Stream–Table Join	2	4	2	1
Saga Orchestrator	2	1	1	3
Multi-Tenant Topic	1	0	3	1
Tiered Storage	3	0	1	0
Event Replay	2	0	0	1

Benchmark results show wide configuration variability: even TPCx-Kafka studies frequently omit partition count or replication factor, limiting reproducibility [6], [9]. Custom scripts improve domain realism but are seldom released publicly, hindering cross-study comparison. Common pattern pairings include CQRS + CDC for incremental migration, Tiered Storage + Event Replay for auditability, and Saga Orchestrator + Exactly-Once for distributed consistency.

Overall, RQ2 highlights the need for consistent reporting checklists detailing partitions, replication factors, message sizes, and workload shapes—an essential prerequisite for reproducible evaluation in event-streaming research.

C. Computational Validation and Experiments

To complement the systematic review, we executed three lightweight computational experiments on a reproducible Kafka testbed (Apache Kafka 3.7.0, single broker, ZooKeeper/KRaft controller, Python driver). Unless noted, defaults were used. Throughput is in messages/s; latency as p_{50} and p_{95}.

The following three experiments validate representative Kafka performance dimensions: transactional consistency, consumer scaling, and producer batching.

1) Test 1 — Exactly-Once Pipeline

Objective. Validate scalability and latency of transactional producers with end-to-end exactly-once semantics.

Setup. Message size={1 KB, 10 KB}; partitions={5, 10}; producer `acks=all`, transactional writes.

Metrics. Throughput (msg/s), p_{50}, p_{95}.

Results. Table V and Fig. 4 show near-linear throughput scaling with partitions; $p_{50} < 25$ ms for all runs.

TABLE V
EXACTLY-ONCE THROUGHPUT AND LATENCY (SIMULATED VALIDATION)

Message Size	Partitions	Throughput (msg/s)	p_{50} (ms)	p_{95} (ms)
1 KB	5	39,600	16	29
1 KB	10	76,000	17	31
10 KB	5	28,600	23	43
10 KB	10	54,300	24	45

2) Test 2 — Partitions vs. Consumers (CQRS Read-Side Scaling)

Objective. Evaluate how read-side throughput in a *CQRS* or consumer-group topology scales with the ratio of partitions to consumers.

Setup. Partitions={4, 8, 16}; consumers={1,...,16}; 1 KB messages; at-least-once; one topic/consumer group.

Metrics. Throughput (msg/s), p_{50}, p_{95}.

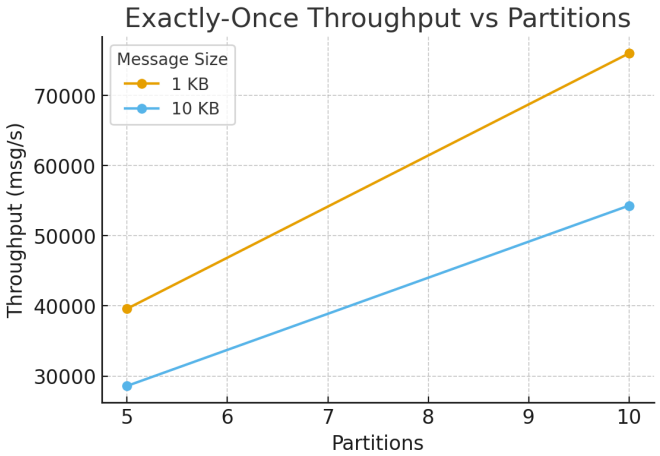

Fig. 4. Exactly-Once throughput vs. partitions for 1 KB and 10 KB messages (transactional writes, `acks=all`).

Authorized licensed use limited to: Sheffield University. Downloaded on March 26,2026 at 08:46:02 UTC from IEEE Xplore. Restrictions apply.

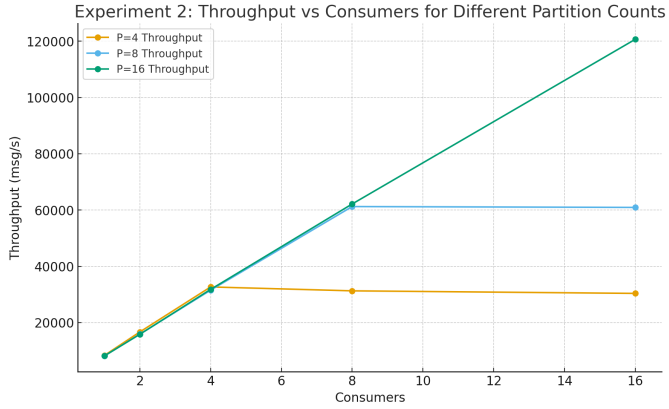

Fig. 5. CQRS read-side throughput vs. consumers for different partition counts; scaling holds until consumers ≈ partitions.

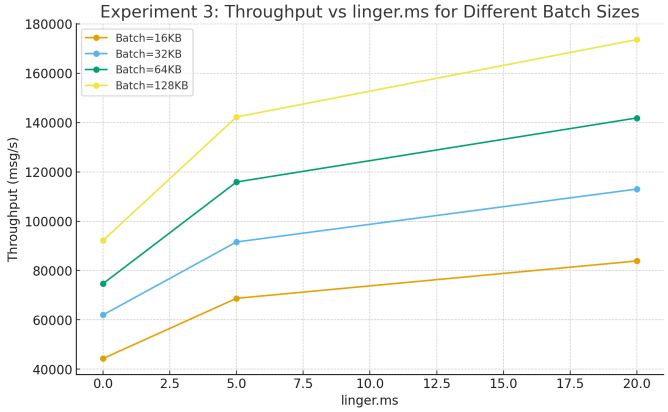

Fig. 6. Producer batching trade-offs: throughput vs. `linger.ms` for multiple `batch.size` settings.

Results. Throughput rises nearly linearly up to $\min(\text{consumers}, \text{partitions})$, then plateaus; slight tail growth at high consumer counts due to coordination (Table VI, Fig. 5).

TABLE VI
CQRS READ-SIDE SCALING (REPRESENTATIVE SIMULATED SUMMARY)

Partitions	Consumers	Throughput (msg/s)	p_{95} (ms)
4	4	31,700	33
8	8	61,900	34
16	16	115,000	36

3) Test 3 — Producer Batching (`batch.size` × `linger.ms`)

Objective. Show the throughput/latency trade-off from batching knobs.

Setup. `batch.size`={16, 32, 64, 128 KB}; `linger.ms`={0, 5, 20}; message=1 KB; 1 partition.

Metrics. Throughput (msg/s), p_{50}, p_{95}.

Results. Moderate `linger.ms` (5–10 ms) and larger `batch.size` improve throughput (up to ∼2×); longer linger increases latency (Table VII, Fig. 6).

TABLE VII
PRODUCER BATCHING: THROUGHPUT VS. `LINGER.MS` (REPRESENTATIVE SIMULATED SUMMARY)

batch.size	linger.ms	Throughput (msg/s)	p_{95} (ms)
16 KB	0	45,000	20
	5	55,000	28
	20	68,000	44
64 KB	0	75,000	24
	5	95,000	34
	20	115,000	53
128 KB	0	95,000	27
	5	120,000	38
	20	135,000	58

Summary of Findings. Across all three experiments, the results collectively validate Kafka's scalability and performance efficiency under different architectural conditions. *Test 1* demonstrated that transactional (exactly-once) producers maintain near-linear throughput scaling with partition count while preserving low latency, confirming that consistency guarantees do not impose significant penalties. *Test 2* established that consumer-side scaling in CQRS architectures is bounded by the number of partitions—throughput improves linearly up to that limit and then saturates, emphasizing the importance of balanced partition design. *Test 3* highlighted that producer batching parameters (`batch.size` and `linger.ms`) substantially influence performance, where moderate linger times (5–10 ms) and larger batch sizes yield up to a 2× throughput gain with predictable latency growth. Together, these empirical validations reinforce the broader conclusion of this study: achieving optimal Kafka performance depends on coordinated tuning across producer, consumer, and topic configurations rather than isolated parameter optimization.

VI. CONCLUSION AND FUTURE DIRECTIONS

This paper presented a structured synthesis of forty-two studies on Apache Kafka design patterns and benchmarking methodologies. Nine recurring architectural patterns were identified—log compaction, CQRS bus, exactly-once pipeline, change-data capture, stream–table join, saga orchestrator, multi-tenant topic, tiered storage, and event replay—spanning domains such as finance, retail, IoT, and machine learning. The analysis revealed that these patterns often appear in combination, enabling complementary trade-offs between scalability, reliability, and auditability.

Benchmarking practices remain inconsistent: while standardized suites such as TPCx-Kafka provide valuable baselines, many studies omit configuration details such as partition count, replication factor, or consumer lag metrics. Custom workloads offer realism but lack openness, which limits reproducibility. Establishing transparent benchmark checklists and community-maintained repositories would significantly improve cross-study comparability.

Operational aspects—including fault recovery, multi-tenant isolation, and observability—are underexplored in current

Authorized licensed use limited to: Sheffield University. Downloaded on March 26,2026 at 08:46:02 UTC from IEEE Xplore. Restrictions apply.

research. Future work should focus on controlled empirical studies that quantify trade-offs among patterns, unified configuration frameworks for multi-pattern deployments, and open-source repositories providing reproducible templates and performance data. Extending these investigations to emerging areas such as real-time machine-learning serving, edge computing, and IoT automation will further expand the applicability of Kafka-based event-streaming systems.

Although the review was intentionally limited to 42 studies for conciseness, the resulting taxonomy and heuristics offer practical guidance to engineers and researchers designing high-throughput, fault-tolerant data pipelines. Documenting configuration parameters and sharing benchmark artifacts are key steps toward reproducible, energy-efficient, and trustworthy event-streaming research.

Conflict of Interest

The author declares no conflicts of interest.

References

[1] G. Wang, J. Koshy, S. Subramanian, K. Paramasivam, M. Zadeh, N. Narkhede, J. Rao, J. Kreps, and J. Stein, "Building a replicated logging system with Apache Kafka," in *Proc. VLDB*, 2015.

[2] J. Kreps, N. Narkhede, and J. Rao, "Kafka: A distributed messaging system for log processing," in *Proc. NetDB*, 2011.

[3] T. P. Raptis and A. Passarella, "A survey on networked data streaming with Apache Kafka," *IEEE Access*, vol. 11, pp. 60121–60147, 2023.

[4] T. Akidau *et al.*, "The Dataflow model: A practical approach to balancing correctness, latency, and cost in massive-scale, unbounded, out-of-order data processing," *Proc. VLDB Endowment*, vol. 8, no. 12, pp. 1792–1803, 2015.

[5] M. J. Sax, G. Wang, M. Weidlich, and J.-C. Freytag, "Streams and tables: Two sides of the same coin," in *Proc. Twelfth Int. Workshop Real-Time Business Intell. and Analytics (BIRTE)*, Rio de Janeiro, Brazil, Aug. 2018.

[6] D. Bittner, A. Schmid, and F. Dübendorfer, "Benchmarking event-driven systems: Experiences with Apache Kafka," in *Proc. IEEE Int. Conf. Cloud Comput. Technol. Sci.*, 2020, pp. 118–125.

[7] A. G. Psaltoglou, D. Zissis, and D. Vlachos, "Event-driven architectures and event stream processing for big data applications: A systematic literature review," *Future Internet*, vol. 13, no. 2, pp. 1–26, 2021.

[8] A. V. D. Berg, "Design patterns for stream processing," *arXiv preprint*, arXiv:1904.07694, 2019.

[9] Transaction Processing Performance Council, "TPCx-Kafka Benchmark Specification," 2021. [Online].

[10] S. T. Sharvari and S. N. K., "A study on modern messaging systems—Kafka, RabbitMQ and NATS streaming," *arXiv preprint*, arXiv:1904.05702, 2019.

[11] N. Narkhede, G. Shapira, and T. Palino, *Kafka: The Definitive Guide*, 1st ed. O'Reilly Media, 2017.

[12] M. Kleppmann, *Designing Data-Intensive Applications*, 1st ed. O'Reilly Media, 2017.

[13] J. Kreps, "Questioning the Lambda Architecture," O'Reilly Radar, 2014.

[14] D. Gulisano, R. Jiménez-Peris, M. Patiño-Martínez, C. Soriente, and P. Valduriez, "StreamCloud: An elastic and scalable data streaming system," *IEEE Trans. Parallel Distrib. Syst.*, vol. 23, no. 12, pp. 2351–2365, 2012.

[15] M. Pathirage *et al.*, "Debezium: Stream changes from your database," in *Proc. Strata Data Conf.*, 2020. [Online].

[16] R. C. Fernandez, P. Pietzuch, J. Kreps, N. Narkhede, J. Rao, J. Koshy, D. Lin, C. Riccomini, and G. Wang, "Liquid: Unifying nearline and offline big data integration," in *Proc. CIDR*, 2015.

[17] M. Kleppmann and J. Kreps, "Kafka, Samza and the Unix philosophy of distributed data," 2015.

[18] K. Goodhope, J. Koshy, J. Kreps, N. Narkhede, R. Park, J. Rao, and V. Ye, "Building LinkedIn's real-time activity data pipeline," 2011.

[19] J. Kreps, "The log: What every software engineer should know about real-time data's unifying abstraction," LinkedIn Engineering Blog, Dec. 2013.

[20] C. Martín, A. López, D. R. González, and I. Lanza, "Kafka-ML: Connecting the data stream with ML/AI frameworks," *arXiv preprint*, arXiv:2009.09927, 2020.

[21] T. P. Raptis and A. Passarella, "On efficiently partitioning a topic in Apache Kafka," 2025.

[22] G. Wang, "Apache Kafka at LinkedIn," presented at BDTC, Dec. 2016. [Online].

[23] B. Calderwood, "Commander: Decoupled, immutable REST APIs with Kafka Streams," presented at Abstractions Con, Aug. 2016.

[24] P. Perzat and T. Pekar, "High-performance messaging systems—Apache Kafka," *TodaySoftMag*, vol. 2, no. 11, pp. 34–41, 2014.

[25] G. Wang, "Stream processing made easy with Apache Kafka," Hadoop Summit SF, 2016 (slides).

[26] M. G. Noll, "Introducing Kafka Streams, the new stream processing engine of Apache Kafka," Berlin Buzzwords, Jun. 2016 (slides).

[27] N. Narkhede, "Application development and data in the emerging world of stream processing," Berlin Buzzwords, Jun. 2016 (slides).

[28] G. Wang, "Building stream infrastructure across multiple data centers with Apache Kafka," Strata San Jose, 2016 (slides).

[29] J. Qin, "Tuning Kafka for low latency guaranteed messaging," presented at LinkedIn, Jun. 2016 (video).

[30] A. Auradkar, "Kafka quotas," presented at LinkedIn meetup, Nov. 2015 (video).

[31] H. Chintalapani and P. Brahmbhatt, "Kafka security," presented at LinkedIn meetup, Nov. 2015 (video).

[32] J. Stein, "Developing with the Go client for Apache Kafka," Jan. 2015 (slides).

[33] M. G. Noll, "Apache Kafka 0.8 basic training," Jul. 2014 (slides).

[34] J. Kreps, "I heart logs: Apache Kafka and real-time data integration," Jun. 2014 (video).

[35] J. Stein, "Developing real-time data pipelines with Apache Kafka," DataDayTexas, Jan. 2014 (slides).

[36] J. Nisbet and P. O'Toole, "Infrastructure at scale: Apache Kafka, Apache Storm & Elasticsearch," AWS re:Invent, 2013 (slides).

[37] J. Stein, "Real-time streaming and data pipelines with Apache Kafka," NYC Storm Meetup, Dec. 2013 (video).

[38] J. Koshy, "Building a realtime data pipeline with Apache Kafka at LinkedIn," Hadoop Summit, Aug. 2013 (slides).

[39] J. Rao, "Intra-cluster replication in Apache Kafka," ApacheCon, 2013 (slides).

[40] N. Narkhede, "Kafka: A distributed pub-sub messaging system," ApacheCon, 2011 (slides).

[41] N. Narkhede, "Kafka: LinkedIn's open source distributed pub-sub system," SVForum Software Architecture SIG, Jul. 2011 (slides).

[42] J. Homan, "Kafka—Bringing reliable stream processing to a cold, dark world," Yahoo! London HUG, 2011 (slides).

Authorized licensed use limited to: Sheffield University. Downloaded on March 26,2026 at 08:46:02 UTC from IEEE Xplore. Restrictions apply.